

```

        break
    except StopIteration, e:
        return None
    except Exception, e:
        pass # do nothing if not an image
    self.image = image.resize(self.new_size)
    return self.image

```

You will get the next file, but since not all files in the directory may be images, you need to ignore the other files by using exception handling and iterating till you find an image.

Image.open will create an image object from the file but will raise an exception if the file does not contain a valid image. The method, *Resize*, on the *Image* object will create a new image of the desired size. You can find out more about what you can do with the python imaging module at www.pythonware.com/library/pil/handbook/index.htm.

Finally, return the resized image; but if there are no more files, return a null value.

Now, examine the code used to add a caption to the image:

```

def photo_with_caption(self, caption):
    self.im_caption = self.image.copy()
    draw = ImageDraw.Draw(self.im_caption)
    draw.text((50, self.new_size[1] - 50), caption)
    return self.im_caption

```

Since you may wish to change the caption, you can work with the copy of the image. The *ImageDraw* module of the imaging library allows you to draw on the image object. In this case, you are drawing some text on the image. The position chosen is 50 pixels from the left and bottom edges. The revised image is returned, as you will see, to the GUI object, which will display it.

```

def save(self):
    try:
        self.im_caption.save('save/' + self.file_name)
    except Exception, e:
        self.image.save('save/' + self.file_name)

```

The *save* method will save the image with the caption with the same name as the existing file, but in the *save/* directory. However, if one has not been created, it will save the resized image.

Interactive transformations

Now, you will need a GUI class to use the above class. The GUI should show you one image and allow you to add a caption to it. Once you are satisfied, save the photograph and move on to the next one. Or, you may decide to skip a photograph.

```
import Tkinter
```

```

class gui:
    def __init__(self, photos):
        # Save the photo application object context
        self.photos = photos
        self.root = Tkinter.Tk()

        # The photo frame
        self.foto = Tkinter.Canvas(self.root)
        self.foto.pack()

        # The application interaction frame
        self.frame = Tkinter.Frame(self.root)
        self.frame.pack()

        # Text caption
        self.caption = Tkinter.Entry(self.frame, width=72)
        self.caption.pack(side=Tkinter.LEFT)
        self.caption.insert(Tkinter.END, 'Press Enter to Apply Caption')
        self.caption.bind('<Return>', self.apply_caption)

        # The Buttons
        self.save = Tkinter.Button(self.frame, text='Save and Next',
            command=self.save_and_next)
        self.skip = Tkinter.Button(self.frame, text='Next', command=self.
            next_image)
        self.save.pack(side=Tkinter.LEFT)
        self.skip.pack(side=Tkinter.LEFT)

        # Show the first image and start the event loop
        self.next_image()
        self.root.mainloop()

```

Your GUI consists of two parts -- a canvas on which the photograph will be displayed and a frame for interacting with the application.

The frame has a text entry widget and two buttons. The *Save* and *Next* button will save the current image and display the next one. The *Skip* button will just display the next image.

The text entry widget is triggered by the *Return* or the *Enter* key to copy the text you have entered onto the image.

Incidentally, it is not appropriate to import all from Tkinter (that is, *from Tkinter import **) because Tkinter also has a class *Image*, which will conflict with the import of the *Image* module.

The rest of the code in the GUI class will be as follows:

```

def display_image(self, image):
    self.foto['width'] = image.size[0]
    self.foto['height'] = image.size[1]
    self.tk_image = ImageTk.PhotoImage(image)
    self.foto.create_image(0,0,anchor='nw',image=self.tk_image)

```

The method to display the image changes the size of the photo frame to the size of the image. The *ImageTk* module in Python imaging is used to convert the image object into an image object for Tkinter. The *Create image* method of the canvas displays the image.

```
def next_image(self):
```